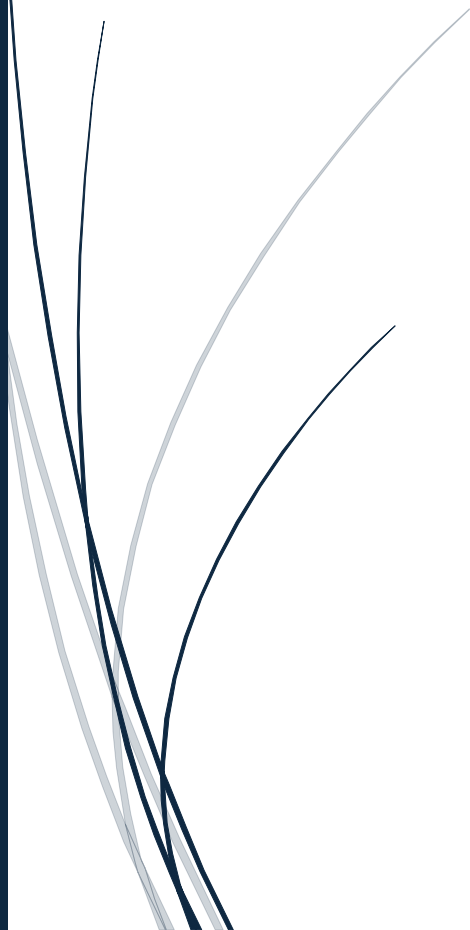




[Datum]

[Titel van document]

[Ondertitel van document]



[naam]
[BEDRIJFSNAAM]

Inhoud

Geen inhoudsopgavegegevens gevonden.

Voorwoord

In dit onderstaande document vindt u alle informatie over [project] beveiligings maatregelen

Doel van de beveiliging

We beveiligen de applicatie zodat:

- Alleen de juiste mensen kunnen inloggen en specifieke dingen doen.
- Gegevens zoals accounts, persoons gegevens, bestellingen of dergelijken niet zomaar ingezien of aangepast kunnen worden.
- De applicatie betrouwbaar blijft voor niets wetende eindgebruikers als een kwaadwillend persoon iets probeert.
- Kleine fouten niet kunnen leiden tot data lekken.

Wat beschermen wij?

- Account gegevens
- Persoons gegevens
- Belangrijke acties(betalen, bestel geschiedenis, gegevens wijzigen)
- Bedrijfsdata (api-key, admin functies,)

Typische risico's

Wat zijn typische beveiliging risico's binnen een software applicatie?

- Wachtwoorden gokken/brute forcen.
- CSRF-attacks.
- SQL-injectie.
- Incorrecte Autorisatie voor pagina's of bestanden(bijv. admin pagina's).
- Onveilig loggen of onjuist loggen van gebeurtenissen of informatie.
- Overbelasting van servers.

Wat doen wij?

Welke maatregelen zullen wij nemen om ervoor te zorgen dat dit geen risico's kunnen zijn binnen de applicatie?

Wachtwoorden:

wachtwoorden worden gehashed opgeslagen door middel van bcrypt hash in Laravel. Op deze manier zijn de wachtwoorden vrijwel nooit meer te dehashen. Een password reset wordt gedaan met een limited time token die via de mail verstuurd word naar de gebruiker en heeft een 1 time use. Wachtwoorden worden ook met minimale eisen opgesteld zodat ze niet te makkelijk te gokken zijn.

CSRF-Attacks:

CSRF attacks staat voor Cross-Site Request Forgery. Dit houdt in dat een andere website die een gebruiker bezoekt, de browser een request kan laten doen naar een andere applicatie waar die gebruiker al is ingelogd. De browser stuurt daarbij automatisch de sessiecookies mee, waardoor de applicatie kan denken dat de gebruiker zelf de actie uitvoert (bijvoorbeeld een wijziging in accountgegevens).

Via een CSRF-token kunnen we dit voorkomen. De server verwacht bij zo'n request een unieke, onvoorspelbare token die bij de sessie (of het formulier) hoort en die door de client moet worden meegestuurd. Een externe website heeft die token normaal gesproken niet, waardoor een vervalst request zonder de juiste token wordt afgewezen.

SQL-Injectie:

Wij gebruiken laravel eloquent om ervoor te zorgen dat query's automatisch parameter binding krijgen. Alle input velden en dergelijken worden altijd gevalideerd. Op deze manier kan een user geen SQL-query invoeren en zo data terug krijgen.

Authorisatie:

Wij zullen Auth functies gebruiken om ervoor te zorgen dat je niet zomaar bij pagina's kan komen waar je nooit bij hoefde te komen. We checken altijd bij belangrijke acties of de juiste gebruiker is ingelogd bij elke stap die gedaan kan worden.

Onveilig loggen:

Onveilig loggen betekent dat je per ongelijk gevoelige informatie in de logs zet of dingen niet in de log zet die je wel erin wilt hebben. We loggen geen wachtwoorden, tokens, api-keys of andere mogelijke gevoelige data. We loggen wel gebeurtenissen zoals inloggingen, mislukte inlogmomenten, admin acties en eventueel andere acties. Logs zijn niet openbaar bereikbaar alleen met specifieke toegang kun je die inzien.

Overbelasting van de servers:

Wij zullen "rate limiting" gebruiken om ervoor te zorgen dat de server niet overbelast kan raken door een grote hoeveelheid requests. Als iemand over het limiet gaat krijgt hij daar

een melding van en wordt de functie tijdelijk uitgeschakeld gebaseerd op IP-adressen en/of user id.